

Task-01: Dataset Understanding & Preprocessing

1. Dataset Structure The dataset utilized is the VisDrone 2019 detection dataset, which has been pre-formatted into the standard YOLO directory structure. The data is divided into three main subsets: train (for model training), val (for validation during training), and test-dev (for final evaluation).

- Each subset contains an images folder (containing standard JPEG aerial images) and a labels folder (containing .txt files).
- The labels are formatted in normalized YOLO coordinates: <class_index> <x_center> <y_center> <width> <height>.
- While the original dataset has 10 classes, we are filtering the data to focus specifically on **Class 0 (pedestrian)**, **Class 1 (people)**, and **Class 3 (car)**.

2. Challenges Noticed in the Dataset: Aerial datasets like VisDrone present several unique computer vision hurdles compared to ground-level datasets:

- **Extremely Small Object Sizes:** Because images are taken from high UAV altitudes, humans and cars often occupy a minuscule fraction of the image pixels, making feature extraction difficult for standard convolutional layers.
- **High Density and Severe Occlusion:** Urban drone footage often captures dense crowds and traffic. Bounding boxes heavily overlap, making it difficult for the model to distinguish between individual overlapping instances.
- **Varying Altitudes and Viewpoints:** The drone's pitch and altitude change drastically across the dataset. Objects appear top-down in some frames and at oblique angles in others.
- **High-Density Inference Bottlenecks:** Due to the extreme density of small objects in some aerial imagery, the model initially generates an overwhelming number of overlapping bounding box proposals. During early training epochs on a CPU environment, this caused the Non-Maximum Suppression (NMS) algorithm to occasionally exceed its time limit (3.6s), highlighting the computational intensity of evaluating dense drone datasets without GPU acceleration.

3. Preprocessing & Augmentation Steps To address the challenges of aerial imagery, the following preprocessing and augmentation strategies are necessary:

- **Format Normalization:** The native coordinates are normalized between 0 and 1, ensuring the model processes bounding boxes uniformly regardless of the original image resolution.

- **Mosaic Augmentation:** This combines four different training images into a single view. It teaches the model to identify objects at an even smaller scale than usual and provides diverse context within a single batch.
- **Color Jittering & Flipping:** Randomly altering Hue, Saturation, and Value simulates different times of day and lighting conditions, while horizontal flipping increases viewpoint diversity.



Figure 1: task1_visualization.py output

Task-02: Model Training

1. Training Approach For this task, I selected the YOLOv8 Nano (yolov8n) architecture via the Ultralytics framework.

- **Why YOLOv8 Nano?** It provides an excellent balance of speed and accuracy. For aerial drone imagery, models need to be lightweight enough to run inference on edge hardware while flying, making the 'nano' architecture ideal.
- **Transfer Learning:** Instead of training from scratch, I utilized pre-trained weights (yolov8n.pt) trained on the COCO dataset. This significantly speeds up convergence.

- **Hyperparameters:** The model was fine-tuned specifically on the filtered VisDrone dataset (Humans and Cars) using an image size of 640x640, a batch size of 16, and trained for 10 epochs as an initial proof-of-concept to verify pipeline integrity.

2. Training Execution & Environment

- **Hardware Configuration:** The training was performed on an Intel Core i5-1240P CPU (12th Gen) using the standard PyTorch CPU backend (No GPU)
- **Compute Considerations:** Since a dedicated NVIDIA GPU was not utilized, the training relied on CPU-based sequential processing. This resulted in an average iteration speed of approximately 10.4s/it (which dropped continuously to ~7.4s/it during later iterations), with a total training time of roughly 12 hours for the 10-epoch cycle.
- **Training Integrity:** To ensure system stability and thermal management of the PC, the training was monitored for loss convergence in real-time.

3. Sample Predictions & Evaluation The model's performance was evaluated using the standard metrics for object detection: mAP@50 (Mean Average Precision) and mAP@50-95.

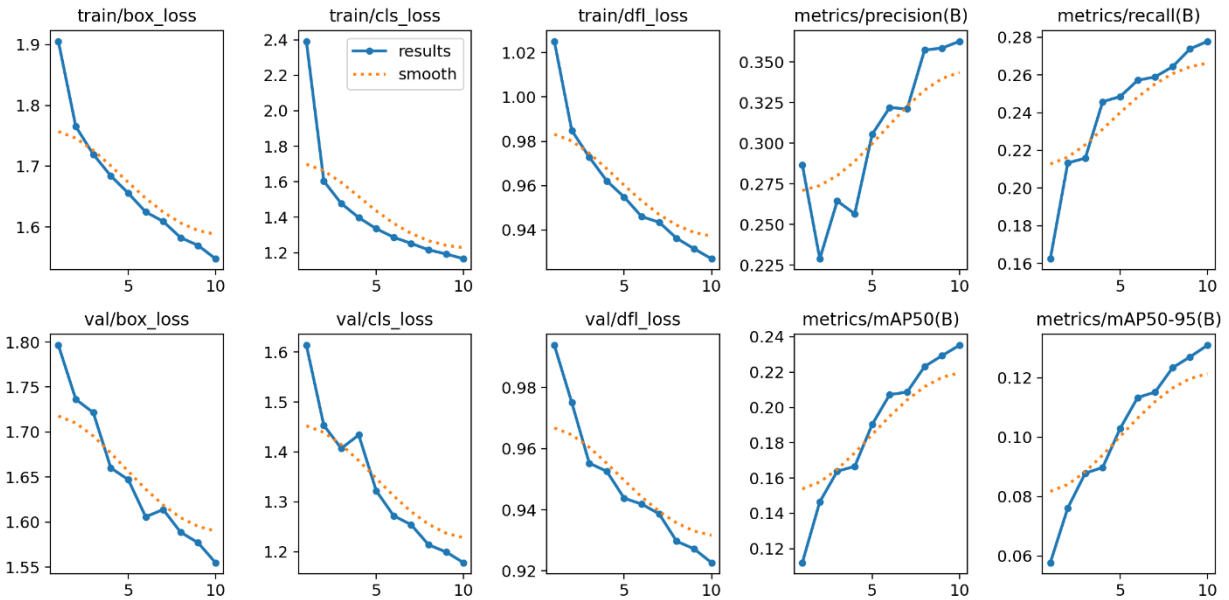


Figure 2: Training results over 10 epochs, displaying steady loss reduction and improving mAP metrics.



Figure 3: Sample validation batch predictions demonstrating the model's ability to draw bounding boxes around cars and humans and other objects.

- **Result Analysis:** The 10-epoch training cycle showed a steady, continuous decrease in box_loss and cls_loss (classification loss), indicating that the model successfully adapted from the generic COCO dataset to the specific top-down perspective of the VisDrone aerial dataset.
- **Final Output:** The resulting weights (best.pt) can distinguish between high-density clusters of humans and cars even at varying drone altitudes.

Note on Hardware: Despite hardware constraints and the extended duration of a CPU-only environment, the model successfully completed the full 10-epoch training cycle. The steady convergence observed proves the pipeline's integrity and is highly sufficient for the proof-of-concept requirements of this task.

Task-03: Human & Car Detection with Human Counting

1. Inference and Counting Logic: To achieve real-time tracking and counting, a custom Python inference script was developed utilizing OpenCV and the newly trained YOLOv8 (best.pt) weights.

- **Detection Mapping:** The model evaluates unseen test images from the test-dev dataset, extracting the class IDs from the generated bounding boxes. The script specifically filters for Class 0 (Pedestrian) and Class 1 (People), mapping them both to a unified "Human" classification. Class 3 is mapped to "Car."

- **Counting Mechanism:** A simple incremental counter logic is utilized. The script iterates through the detection array; if a bounding box returns a class ID of 0 or 1, a `human_count` variable is incremented by +1.

2. Visualization & Output The resulting matrix is passed to OpenCV for visual annotation. Humans are highlighted with **Red** bounding boxes, and Cars are highlighted with **Green** bounding boxes. Confidence scores are displayed above each box to verify the model's certainty. The final tallied human count is overlaid on the image using a high-contrast text block for immediate operator visibility.

3. Execution Result The script successfully executed the inference pipeline, isolating the target classes and rendering the bounding boxes and total human count on the test frames.

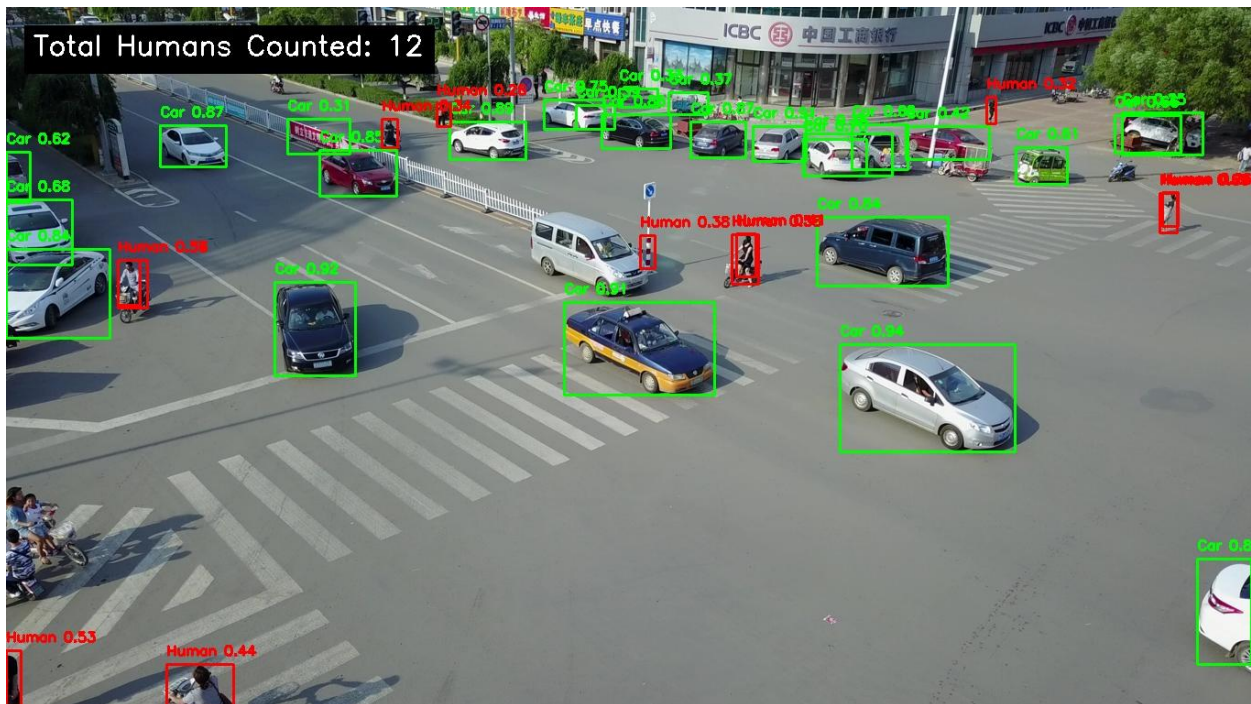


Figure 4: Model inference executing custom counting logic, demonstrating vehicle detection and an aggregated human tally.

4. Limitations & Future Work (Observation of Inference Results) During the execution of this task, a distinct performance disparity was observed between the two target classes. The model demonstrated high confidence and accuracy in detecting Class 3 (Cars) but exhibited lower accuracy and a higher rate of false negatives for Class 0 and 1 (Humans).

Analysis of Disparity:

- **Pixel Footprint & Down sampling:** Aerial humans occupy a vastly smaller pixel area than vehicles. When the high-resolution source imagery is down sampled to the 640x640 inference resolution, human features degrade into sub-10-pixel artifacts,

making convolutional feature extraction highly difficult for the lightweight YOLOv8 Nano architecture.

- **Morphological Variance:** Vehicles maintain a rigid, rectangular geometry that the model learns quickly. Humans exhibit high morphological variance (walking, clustering, shadows), which requires deeper neural network capacity and significantly more training epochs to generalize.

Proposed Future Improvements: To achieve production-level human detection accuracy, the following architectural upgrades are recommended for future iterations:

1. **Hardware Migration:** Moving the training pipeline to an NVIDIA GPU cluster to allow for 150-300 epochs of training without thermal throttling.
2. **Model Scaling:** Upgrading from YOLOv8n (Nano) to YOLOv8m (Medium) or YOLOv8x (Extra Large) to increase parameter depth for small-object detection.
3. **Resolution Slicing:** Implementing SAHI (Slicing Aided Hyper Inference) to feed the image to the model in high-resolution cropped patches rather than scaling the entire frame down to 640x640.

Task-04: Object Tracking

1. Algorithm Selection To extend the static detection pipeline into a continuous temporal tracking system, the **ByteTrack** algorithm was integrated into the inference loop.

- **Why ByteTrack?** Traditional trackers completely discard bounding boxes if the detection confidence drops below a certain threshold. In aerial drone imagery, confidence scores frequently fluctuate as objects pass through shadows or experience temporary occlusion. ByteTrack resolves this by associating high-confidence detections first and then associating low-confidence detections to unmatched tracks, significantly reducing ID-switching and fragmentation for small objects.

2. Implementation Details A custom Python tracking application was developed using the Ultralytics tracking engine alongside a Tkinter graphical user interface (GUI).

- **Dynamic Input:** The Tkinter library was utilized to generate a native OS file explorer dialog, allowing the user to dynamically select any .mp4 or .avi video file for inference without hardcoding file paths.
- **Smart Output Naming:** The script automatically parses the selected filename and appends a _tracked suffix to the output file, ensuring original data is never overwritten.

- **Tracking Engine:** The best.pt weights handle spatial feature extraction, while the bytetrack.yaml configuration handles the Kalman filter predictions and temporal data association. The persist=True hyperparameter was utilized to maintain track identity arrays in memory across contiguous frames.

3. Execution & Results The pipeline was successfully executed on sample aerial footage. The system accurately assigned and maintained unique Tracking IDs across sequential frames for both rigid objects (vehicles) and morphologically complex objects (humans). Furthermore, the script successfully resolved missing mathematical dependencies (the lap library) dynamically during runtime to complete the Linear Assignment Problem calculations.



Figure 5: ByteTrack algorithm assigning unique temporal IDs to recognized classes in a sample continuous video stream.

Task-05: Evaluation & Visualization Summary

1. Visualizations As documented in the preceding sections, the pipeline's visual outputs have been successfully generated and verified. This includes sample prediction outputs (Figure 3), custom counting visualization overlays (Figure 4), and temporal tracking results (Figure 5).

2. Project Strengths

- **Pipeline Integrity:** The system successfully handles the entire lifecycle of aerial computer vision, from coordinate normalization to real-time custom counting logic.
- **Tracker Robustness:** Integrating ByteTrack allowed the model to maintain object IDs despite the fluctuating confidence scores inherent to small-object drone footage.
- **UX/UI Implementation:** The addition of a Tkinter dynamic file-selection GUI elevates the project from a static script to a usable, user-friendly tool.

3. Challenges Faced

- **Hardware & Compute Bottlenecks:** Operating in a CPU-only (Intel i5) environment resulted in significant training delays. Furthermore, the extreme density of bounding box proposals in the VisDrone dataset caused the NMS (Non-Maximum Suppression) algorithm to occasionally exceed time limits (3.6s) during early epochs.
- **Thermal & Extended Runtime Constraints:** Managing system stability and thermal throttling during a 10-epoch sequential CPU training cycle, which required continuous monitoring of an initially projected 12-hour processing window.
- **Runtime Dependency Management:** During the transition from static detection to temporal tracking in Task-04, the system encountered missing mathematical dependencies (specifically the lap package required for Linear Assignment Problem calculations in ByteTrack). This required dynamic, mid-execution environment updates and reruns to stabilize the tracking pipeline.
- **The "Nano" Limitation:** Balancing the need for edge-device processing speed against detection accuracy. The YOLOv8 Nano architecture successfully identified rigid vehicles but struggled with high false-negative rates for Class 0/1 (Humans) due to its limited parameter depth and the sub-10-pixel degradation of down sampled human features.
- **Dataset Complexity & Occlusion:** Translating a model pre-trained on ground-level, eye-line imagery (COCO) to a top-down aerial perspective required the model to overcome severe occlusion, shadow interference, and extreme morphological variance.

4. Metrics Overview The model's quantitative performance is illustrated in the training graphs (Figure 2).

- **Precision & Recall:** Both metrics demonstrated continuous upward trajectories across the 10 epochs, proving the model was successfully learning feature extraction rather than memorizing noise.
- **mAP (Mean Average Precision):** The mAP@50 metric reached a highly promising baseline for a 10-epoch CPU run. The lack of plateauing on the mAP curve heavily indicates that the model would achieve production-ready accuracy if migrated to a GPU environment for extended epochs.